

1. Criterio general de diseño

- El proyecto trabaja con varias fuentes de datos musicales con estructuras diferentes: un dataset emocional etiquetado, un catalogo de tracks de Spotify y un dataset enriquecido con letras. Por ese motivo, la arquitectura no se basa en una unica base de datos tradicional, sino en un data lake por capas que permite conservar los datos originales, transformarlos progresivamente y preparar datasets finales para Machine Learning.
- La decision principal es usar Amazon S3 como repositorio unico del proyecto. Alrededor de S3 se integran servicios especializados: Kafka para ingesta en tiempo real, Spark para transformacion, Glue/Athena para catalogacion y consulta, RDS para trazabilidad estructurada, MongoDB Atlas para metadatos flexibles y Lambda para automatizacion basada en eventos.

2. Amazon S3 como repositorio unico del data lake

- Amazon S3 se elige como almacenamiento principal porque el proyecto necesita guardar datos heterogeneos en diferentes estados de procesamiento:
 - Bronze: eventos crudos recibidos desde Kafka en formato JSONL.
 - Silver: datos limpios, tipados, normalizados y deduplicados en Parquet.
 - Gold: datasets preparados para entrenamiento, validacion, clasificacion y recomendacion.
- S3 encaja mejor que una base de datos relacional como almacenamiento principal porque no obliga a fijar un esquema unico desde el inicio. Esto es importante porque las fuentes no tienen la misma estructura: unas contienen etiquetas emocionales, otras metadatos musicales, otras letras y otras características acústicas.
- También permite separar almacenamiento y computo. Los datos quedan persistidos en S3 y pueden ser procesados después por Spark, consultados por Athena o consumidos por notebooks y scripts de entrenamiento sin duplicar innecesariamente la información.

3. Formatos usados: JSONL y Parquet

- En Bronze se utiliza JSONL porque representa bien eventos individuales procedentes de Kafka. Cada linea es un registro independiente, facil de escribir incrementalmente y adecuado para simular ingesta continua.
- En Silver y Gold se utiliza Parquet porque es un formato columnar mas eficiente para analisis y Machine Learning. Reduce espacio, conserva tipos de datos y permite consultas mas rapidas con herramientas como Spark y Athena.

4. Apache Kafka para ingesta en tiempo real

- Apache Kafka se utiliza para simular un escenario realista en el que los datos llegan como eventos y no como un unico archivo estatico. En este proyecto, los CSV originales se publican por lotes en topics Kafka y despues un consumer los escribe en la capa Bronze.
- La eleccion esta justificada porque el recomendador musical podria recibir en un entorno real nuevas canciones, nuevas interacciones de usuarios o nuevos eventos de escucha. Kafka permite desacoplar la fuente de datos del procesamiento posterior: el productor solo publica eventos y el consumidor se encarga de persistirlos.

5. Apache Spark como motor de procesamiento

- Spark/PySpark se usa para transformar Bronze en Silver y preparar datos limpios para fases posteriores. Es adecuado porque el pipeline necesita:
 - tipar columnas numericas;
 - normalizar nombres y formatos;
 - eliminar duplicados;
 - filtrar valores fuera de rango;
 - unificar estructuras diferentes;
 - escribir salidas analiticas en Parquet.
- Aunque el proyecto pueda ejecutarse localmente, Spark representa una tecnologia escalable y coherente con un pipeline de datos real. Si el volumen creciera, la misma logica podria ejecutarse en un entorno distribuido.

6. AWS Glue Data Catalog y Amazon Athena

- Glue Data Catalog se usa para registrar las tablas externas generadas sobre S3. Athena se usa para validar los datos con SQL sin tener que cargarlos en una base de datos adicional.
- Esta decision evita duplicar los datasets Gold/Silver en un motor relacional solo para consultarlos. Athena consulta directamente los Parquet almacenados en S3, lo que encaja con una arquitectura ELT: primero se almacenan los datos en el lake y despues se consultan o transforman segun la necesidad.
- En el proyecto, Athena permite comprobar conteos, esquemas y resultados de integracion de forma reproducible.

7. AWS RDS MySQL para trazabilidad estructurada

- RDS no se utiliza como data lake principal. Se utiliza para guardar informacion estructurada del funcionamiento del pipeline:
 - resumen de ejecuciones;
 - resumen por dataset y capa;

- estadísticas de columnas;
 - estado, tiempos y recuentos.
- Esta información encaja bien en un modelo relacional porque tiene estructura estable y se consulta de forma tabular. RDS permite responder preguntas como: que ejecución se hizo, cuantas filas produjo, que datasets se procesaron y que estadísticas se obtuvieron.

8. MongoDB Atlas para metadatos flexibles

- MongoDB Atlas se utiliza para guardar metadatos con estructura más variable, especialmente información granular de archivos del data lake. Por ejemplo, documentos sobre archivos Parquet, tamaños, row groups, rutas, datasets y capas.
- La elección de MongoDB está justificada porque estos metadatos pueden variar según el tipo de archivo, la capa o la ejecución. En lugar de forzar muchas tablas relacionales, se guardan como documentos flexibles.
- MongoDB complementa a RDS: RDS guarda resumen estructurado y MongoDB guarda detalle flexible.

9. AWS Lambda para automatización y auditoría

- AWS Lambda se integra con eventos de S3 para automatizar tareas cuando se escriben objetos en el data lake. En el proyecto, la función genera eventos de auditoría cuando aparecen objetos o marcadores relevantes en Bronze, Silver o Gold.
- La justificación técnica es que Lambda permite reaccionar a cambios del almacenamiento sin ejecutar manualmente otro proceso. Esto representa un patrón habitual en arquitecturas data lake: S3 actúa como fuente de eventos y Lambda automatiza acciones ligeras de auditoría, validación o notificación.

10. Por qué no usar solo una base de datos

- Usar solo RDS no sería adecuado porque los datasets son heterogéneos, pueden crecer en volumen y tienen fases de procesamiento distintas. Una base relacional obligaría a definir esquemas rígidos demasiado pronto y a cargar datos crudos que no siempre están limpios.
- Usar solo MongoDB tampoco sería ideal porque el entrenamiento y la validación analítica se benefician más de formatos columnares como Parquet y consultas SQL sobre S3.
- Por eso se elige una arquitectura híbrida:
 - S3 conserva los datos principales por capas.
 - Kafka gestiona la entrada de eventos.
 - Spark procesa y normaliza.
 - Glue/Athena cataloga y consulta.
 - RDS registra trazabilidad estructurada.

- MongoDB registra metadatos flexibles.
- Lambda automatiza auditoria basada en eventos.

11. Relacion con el problema de Machine Learning

- La arquitectura soporta directamente el problema de clasificacion emocional y recomendacion contextual. El modelo necesita datos limpios, consistentes y trazables. La capa Gold proporciona las features acusticas y textuales preparadas para entrenamiento, mientras que el catalogo clasificado alimenta la aplicacion de recomendacion.
- Ademas, la separacion por capas permite explicar claramente el linaje de datos: desde las fuentes originales hasta los datasets finales usados por el modelo.

12. Conclusion

- El sistema de almacenamiento elegido no responde solo a cumplir tecnologias del enunciado, sino a una necesidad tecnica del proyecto: integrar multiples fuentes musicales, conservar datos crudos, transformar informacion para Machine Learning, validar resultados con SQL y mantener trazabilidad de ejecuciones.
- Amazon S3 es el repositorio unico del dato. El resto de tecnologias se usan con responsabilidades concretas y complementarias, evitando duplicidades innecesarias y manteniendo una arquitectura escalable, auditable y coherente con un sistema completo de Machine Learning.